Query Optimization

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

You are a database administrator for a social media platform. The platform has a large user base, and users frequently post status updates, comment on posts, and like other users' content. The *posts* table stores user posts, and the *comments* table stores comments on these posts. The marketing team requests a report showing the top 10 most commented posts in the last 30 days, including the number of comments for each post. The query needs to join the *posts* and *comments* tables, filter the results by the *created_at* timestamp, and order the posts by the number of comments. After executing the query, you notice that the performance is slower than expected, especially when the comments table has grown substantially.

1. What are the possible reasons for the query's slow performance when joining the posts and comments tables?

Answer:

The query is slow mainly because the database is doing extra work than required task during processing the data.

```
EXPLAIN
SELECT p.post_id, COUNT(c.comment_id) AS total_comments
FROM posts p
JOIN comments c ON p.post_id = c.post_id
WHERE c.created_at >= NOW() - INTERVAL 30 DAY
GROUP BY p.post_id
ORDER BY total_comments DESC
LIMIT 10;
```

1 Result | 2 Profiler | 3 Messages | 4 Table Data | 5 Info

(Read Only) | Limit rows First row 0 # of rows 1000

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	index	PRIMARY	user_id	4	(NULL)	3	Using index; Using temporary; Using filesort
1	SIMPLE	c	ALL	post_id	(NULL)	(NULL)	(NULL)	5	Using where; Using join buffer (flat, BNL join)

From the given O/P, it can be seen that for the **comments** table the value **type = ALL** and **key = Null** are shown. This means that DB is performing a **Full Table Scan**, where it checks every row in the comments table instead of using an index. Imagine if the table becomes larger the time for full table scan will increase and it will slow down the query.

Another issue is the join operation. The O/P shows **"USING JOIN BUFFER (FLAT, BNL JOIN)"**, which simply means a block nested loop join is being used. This type of joint is not very effective because the database repeatedly compares rows from both the tables, which increases the processing time.

The **EXPLAIN** output also shows **"USING TEMPORARY"** and **"USING FILESORT"** for the **posts** table. This means the database is creating a temporary table and sorting the data manually when applying **GROUP BY** and **ORDER BY.** These extra occupies the memory and slows down the query.

Also, the query identifies data using the **created_at** column for last 30 days. Since no index is being used for this column, the DB has to check each row one by one, which directly slows down the query.

2. How would you optimize the query to speed up the join between the posts and comments tables?

Answer:

To improve the performance of the query, the main aim was to reduce the unnecessary extra work, stop full table scan and make the join more efficient.

1. We are making an index on the **comments** table:

```
CREATE INDEX idx_comments_post_created
ON comments(post_id, created_at);
```

```
/*Index Information*/
---------------------
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
comments	0	PRIMARY	1	comment_id	A	5	(NULL)	(NULL)		BTREE		
comments	1	user_id	1	user_id	A	5	(NULL)	(NULL)		BTREE		
comments	1	idx_comments_post_created	1	post_id	A	5	(NULL)	(NULL)		BTREE		
comments	1	idx_comments_post_created	2	created_at	A	5	(NULL)	(NULL)		BTREE		

This index is essential because it matches the two most important part of query i.e. **post_id** and **created_at.** Now instead of checking whole dataset it will directly go to the necessary record.

2. We are slightly adjusting the query now:

After the indexing part, the query is slightly adjusted so that it starts from the **comments** table. Since this table is filtered by date, it reduces the no. of rows that need to be joined with the **posts** table.

```sql
SELECT
    p.post_id,
    p.content,
    COUNT(*) AS total_comments
FROM comments c
JOIN posts p
    ON p.post_id = c.post_id
WHERE c.created_at >= NOW() - INTERVAL 30 DAY
GROUP BY p.post_id, p.content
ORDER BY total_comments DESC
LIMIT 10;
```

1 Result | 2 Profiler | 3 Messages | 4 Table Data

(Read Only)

post_id	content	total_comments
2	My first ... 17B	3
1	Hello Eve... 15B	2

3. We tested query again using EXPLAIN command:

```sql
EXPLAIN
SELECT
    p.post_id,
    p.content,
    COUNT(*) AS total_comments
FROM comments c
JOIN posts p
    ON p.post_id = c.post_id
WHERE c.created_at >= NOW() - INTERVAL 30 DAY
GROUP BY p.post_id, p.content
ORDER BY total_comments DESC
LIMIT 10;
```

1 Result | 2 Profiler | 3 Messages | 4 Table Data | 5 Info

(Read Only) Limit rows First row 0 # of rows 1000

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	ALL	PRIMARY	(NULL)	(NULL)	(NULL)	3	Using temporary; Using filesort
1	SIMPLE	c	ref	idx_comments_post_created	idx_comments_post_created	4	social_media.p.post_id	1	Using where; Using index

The query was tested again using **EXPLAIN** command, the DB was now using **type = ref** instead of scanning the whole table unlike before.

Lastly, we eradicated the full table scan and implemented indexing which made the query faster and more effective.

3. Would it be beneficial to cache the results of this query for frequently requested reports? If so, how would you implement caching in this scenario?

Answer:

Yes, caching would definitely be useful for this query, especially because it is used for reports that people ask again and again.

In case of this query, it really does a lot of work like joining two tables i.e. **comments** and **posts,** filters the data of around 30 days, and group and sort the results. All of these works take time, it's natural that query will be slow due to increase in the comments. If someone keeps on asking the report, it doesn't make sense to run whole query again and again.

We used both **SQL_N0_CACHE** and **Normal Query**:

```
99    SELECT
100       p.post_id,
101       p.content,
102       COUNT(*) AS total_comments
103   FROM comments c
104   JOIN posts p ON p.post_id = c.post_id
105   WHERE c.created_at >= NOW() - INTERVAL 30 DAY
106   GROUP BY p.post_id, p.content
107   ORDER BY total_comments DESC
108   LIMIT 10;
109
```

1 Result | 2 Profiler | 3 Messages | 4 Table Data

(Read Only)

post_id	content		total_comments
2	My first ...	17B	3
1	Hello Eve...	15B	2

```
88    SELECT SQL_NO_CACHE
89       p.post_id,
90       p.content,
91       COUNT(*) AS total_comments
92   FROM comments c
93   JOIN posts p ON p.post_id = c.post_id
94   WHERE c.created_at >= NOW() - INTERVAL 30 DAY
95   GROUP BY p.post_id, p.content
96   ORDER BY total_comments DESC
97   LIMIT 10;
98
```

1 Result | 2 Profiler | 3 Messages | 4 Table Data

(Read Only)

post_id	content		total_comments
2	My first ...	17B	3
1	Hello Eve...	15B	2

From my own practical test, I ran both the queries. When SQL_N0_CACHE is used, the dataset has to execute all the queries from the scratch every time, which normally takes longer time. When it was removed, the DB can reuse the results, make the queries faster and more effective.

Caching can be implemented using tools like Redis or Memcached to store the results temporarily, or maybe storing results in another table and updating it time by time.

However, caching needs to be managed carefully because the data can become outdated when comments are added in the posts. This can be handled by setting an expiry time or refreshing the cache regularly.

At last, caching reduces the workload on the DB and improvises the performance of frequently asked reports.

Tools and Technology Used:

1. SQLyog

I used **SQLyog** to create database and then implement the required queries. I also extracted the Task3_24071246.sql from it to prove that it was done in **SQLyog**.

2. MS Word

I used **MS Word** to present my answer in more detailed and systemic way. MS Word contains all the **queries**, **outputs**, **screenshots**, etc.